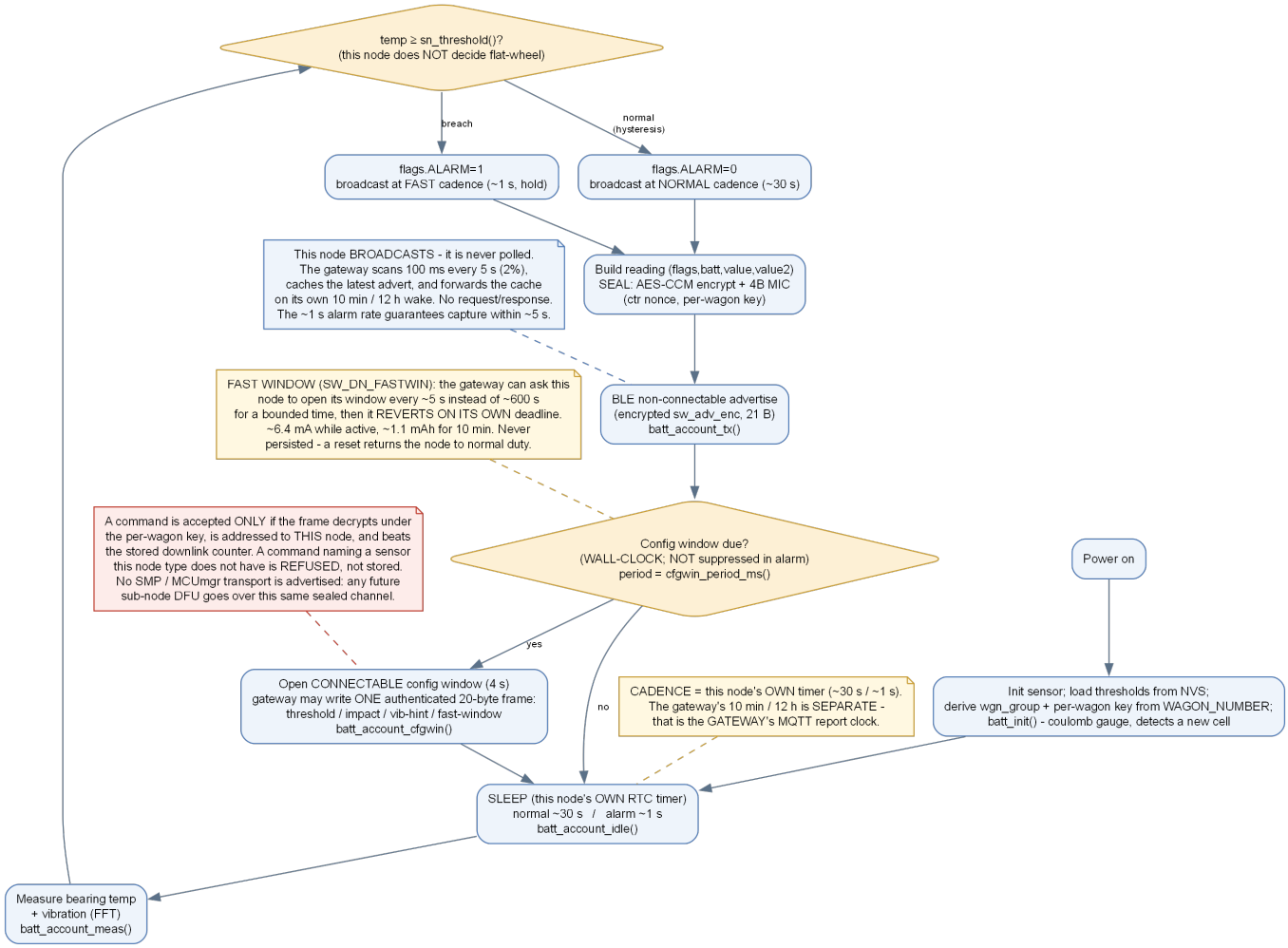
Sub-node — Bearing & Wheel Health

8 per wagon · bearing temperature + vibration signature



NOT IMPLEMENTED IN FIRMWARE. This sheet describes the *target* design. In the current build this node type has no sensor driver: `sensor.c` is a placeholder that returns a fixed 25.0 °C and a zero secondary value, and its overlay declares no I2C bus, so the BMA400 and STS4x fitted to the board are not read. Everything else on this sheet - BLE, AES-CCM, the battery gauge, the config window, the cadence logic - IS implemented and working. Do not field this node type until the sensor driver lands.

1. Role

A battery-powered, **connectionless BLE broadcaster**. It measures its sensor, **encrypts** the reading into a BLE advertisement (AES-CCM, with the wagon-group id for isolation), broadcasts a short burst, and sleeps. On a threshold breach it switches to fast "alarm" advertising so the gateway's receive window catches it within ~5 s. It forms a connection only during a periodic **config window**, and only to receive thresholds.

Timing — no polling: this node advertises on its **own** timer — about every **30 s** normally, **~1 s** while in alarm. The gateway's **10 min / 12 h** cadence is a **separate** clock. This node is **never polled**: it broadcasts blind, the gateway's low-power scan (**100 ms every 5 s, 2% duty**) caches the latest advert, and on the gateway's own schedule it forwards that cache upward.

2. Algorithm

1. Init: load `node_id`, `wgn_group` and thresholds from NVS; start the coulomb gauge (which auto-zeroes if it detects a replacement cell).
2. Wake on the node's own RTC timer; measure bearing temp + vibration (fft).
3. Evaluate against the alarm threshold **with hysteresis** (on temp ≥ 85°C, off < 80°C).

4. **Seal** the reading with AES-CCM under the per-wagon key, using a fresh monotonic counter `ctr` as the nonce (persisted in NVS so it never repeats). Nonce byte 7 carries a direction tag so an uplink nonce can never collide with a downlink one.

5. Advertise the encrypted burst (non-connectable); **slow when normal (~30 s), fast while in alarm (~1 s)**. Both periods are read at runtime from `sn_normal_s()` / `sn_fast_s()`, so a `SW_DN_SET_CADENCE` downlink changes them live and the change survives a reboot (section 5.1). A `DEBUG_TRACE` build seeds **10 s** instead of 30 s so a node is visible on the bench without waiting half a minute per advert - the seed only, not a different mechanism.

6. Every CFGWIN_PERIOD_MS of wall-clock time - **including during an alarm** - open a 4 s connectable **config window** so the gateway can write new thresholds, or stream a firmware image (section 6).

3. Advertisement payload (encrypted `sw_adv_enc`, 21 B)

Cleartext header (company, ver, wgn_group, type, id, ctr) + **AES-CCM ciphertext** of the fields below + 4-byte MIC. The header is authenticated; the reading itself is encrypted.

Field	Meaning for this node
wgn_group	This wagon's isolation group, from the wagon number (gateway drops other groups; cleartext)
node_type / node_id	sensor type + unique id within the wagon (cleartext header)
ctr	monotonic nonce counter, persisted in NVS (cleartext; makes each advert unique)
flags / batt (encrypted)	ALARM/LOWBATT flags, battery % from the coulomb gauge
value	bearing temperature ×10 (°C)
value2	vibration index (valid > 15 km/h)

3.1 Cadence over the air (`SW_DN_SET_CADENCE`, 0x08)

RDSO §7.15 requires the vendor to set operating values in the field, so the advertising period is not fixed at build time. The command carries the **quiet** period in `value` and the **alarm** period in `hyst`, both in **seconds** - the same two wire fields every other threshold uses, so it inherits the individual / by-type / all-nodes fan-out unchanged.

Guard	Behaviour
Range	Both clamped to `CADENCE_MIN_S` 1 s … `CADENCE_MAX_S` 3600 s. Below 1 s buys nothing over the alarm cadence and costs battery; above an hour the node cannot satisfy the gateway's 15-minute staleness check and would be flagged DOWN continuously.
Ordering	The alarm period is never allowed to exceed the quiet one. An alarm reporting *less* often than normal traffic would invert the purpose of the fast cadence.
Persistence	Written to NVS (`sw/norm`, `sw/fast`) and reloaded by `settings_load()`, so a cadence tuned over the air survives both a reboot **and** a firmware update carrying different compile-time defaults.
Effect	Immediate - the very next sleep uses the new value. No reboot.

Battery consequence: the §5.1.20 six-year projection is built on the **30 s** production figure. Dropping the quiet cadence to 5 s multiplies advertising charge by six and invalidates the life estimate - the clamp bounds the damage, it does not prevent it.

4. Alarm / event mapping

Condition	Protocol code	Notes
Bearing over-temp / seizure	HOT_BEARING	node decides; sev red
Wheel flat / RCF (vibration)	FLAT_WHEEL	GATEWAY decides - needs speed > 15 km/h (§7.9)
Cell low	NODE_LOW_BATTERY	advisory, rides next heartbeat

5. Battery gauge (coulomb counting)

The cell is a **non-rechargeable 13 Ah Li-SOCl&sub2; ER34615**. Its discharge curve is flat, so a voltage-to-percent map reads ~100 % for years and then collapses. State of charge is therefore obtained by **integrating what the firmware spends**: every sleep, measurement, advertising burst and config window adds (current × time) in nAh to an accumulator persisted in NVS. Self-discharge is carried as a constant current so elapsed time is never free.

Item	Value
Usable capacity	`BATT_USABLE_MAH` = 10000 (derated from the 13 Ah nameplate for cold and load)
Total current ceiling	13 Ah over a 6-year life = **247 µA average**
Predicted average	~134 µA (sleep 5 + self-discharge 15 + measure 1 + advertising 60 + config window 53)
Independent end-of-life check	`BATT_EOL_MV` = 3000 mV, so counter drift cannot hide a dying cell

Calibration outstanding: every `BATT_I_*_UA` constant except self-discharge is an estimate marked `MEASURE:` in the source. They must be measured with a Nordic PPK2 in series with the cell over one full wake cycle before the projected life means anything.

6. Firmware update over BLE

The node can be updated by **its own gateway only**. There is no SMP service, no MCUmgr, and no phone path - the image arrives as sealed chunks on a second GATT characteristic during the same config window used for thresholds.

Step	What happens
Stage	Gateway downloads the image over HTTP into its NOR `ota` partition and computes a CRC16. The two links are never up together, so the image must be staged before any of it can be sent.
Arm	`SW_DN_OTA_BEGIN` on the config characteristic: erases this node's MCUboot secondary slot and announces the size. Sent **once** per campaign - a resumed transfer must not repeat it.
Stream	192-byte chunks, each its own AES-CCM frame with nonce direction tag `0x02`, accepted only **strictly in order**. Delivery spans as many windows as it takes; progress is persisted in gateway FRAM so a reset resumes rather than restarts.
Verify	`SW_DN_OTA_END` carries the CRC16. A mismatch discards the whole transfer before anything is marked bootable.

Swap	`boot_request_upgrade(BOOT_UPGRADE_TEST)`. **No reboot here** - the node swaps on its own next reset, so an update never interrupts a report in flight. MCUboot verifies the fleet signature and **auto-reverts** if the new image fails to confirm.
Confirm	`boot_write_img_confirmed()`, called only after the new image completes a **full successful cycle** - sense, seal, advertise, account. **TEST mode without this step reverts on the second reboot**, so an update that appeared to install would silently vanish days later. Deferring the confirm until a whole cycle has passed is what makes the auto-revert meaningful: an image that boots but cannot read its sensor is rejected by the node itself.

Three independent gates, in order: per-chunk AES-CCM (a neighbouring wagon or an attacker cannot inject one byte), CRC16 over the whole image (catches truncation or reordering before anything is bootable), and MCUboot signature verification with auto-revert (the final authority). Gate 3 alone would stop unsigned code running, but not stop a wagon spending hours of radio assembling rubbish - that is what 1 and 2 buy.

During a campaign the gateway sends `SW_DN_FASTWIN` so windows open every few seconds instead of every 10 minutes. Fast mode is **deadlined** in the node, so an abandoned campaign or a lost cancel cannot strand it at high duty.

6. Vibration: a hint from the node, a decision by the gateway

The node measures the vibration index into `value2` but **does not raise FLAT_WHEEL**. That test is only valid above 15 km/h, and a bearing node has no accelerometer and no speed input - it cannot tell rolling from shunting, so letting it alarm autonomously would fire during yard handling.

What it does instead: crossing its own `sn_vib_threshold()` sets an internal `VIB_HIGH` flag and drops the advert period from ~30 s to **~1 s**, so the gateway - which knows the speed - gets fresh data within about a second. Before this the node escalated for its own over-temperature alarm but not for a gateway-decided one, so a flat wheel could be reported up to 30× slower than an over-temperature on the same node. RDSO §7.3 asks for a "near real time alert" once a threshold is exceeded.

> Keep the node hint limit **at or below** the gateway's `FLAT_WHEEL_VIB_THRESH`. A node limit above it leaves a band where the gateway would alarm but the node never speeds up, putting the latency straight back. Settable per node: `set_threshold` with `"param":"vib_index"`.

7. What is missing

- Bearing temperature front end - no schematic supplied for this board, so neither the sensor type (NTC / RTD / I2C part) nor its pin is known to the firmware.
- Vibration acquisition: burst-sample the BMA400 and reduce to an index (RMS or FFT band energy). The algorithm and its calibration are not defined anywhere in the current documents.
- Devicetree: no I2C bus, no sensor rail switch, no BMA400 interrupt line in this node's overlay.

Ref: RDSO WD-35-MISC-2024 · SmartWagon Telemetry Protocol Rev.1 (pv=1) · controller nRF54L15 (QFN52). Generated from the firmware sources by DOCS/gen_pdfs.py - regenerate after changing the code.